

Project 1

Predicting Customer Insurance Purchases Using Machine Learning Classification Algorithms

Project By: Pradyot Bilurkar

Abstract

This project aims to predict whether customers will purchase health insurance based on their age and estimated salary. Using a dataset containing user details, we preprocess the data and apply various classification algorithms, including Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Decision Trees, and Random Forest. We evaluate the performance of each algorithm using accuracy, precision, recall, and F1-score metrics. The results are compared to identify the most suitable algorithm for this classification task. The study concludes that Random Forest achieves the best balance between accuracy and generalization. Insights from this project can help insurance companies predict customer behavior and optimize marketing strategies.

Table of Contents

- Introduction
- Literature Review
- Problem Statement
- Data Collection and Preprocessing
- Methodology
- Implementation
- Results
- Discussion
- Conclusion
- References
- Appendices

Introduction

For insurance companies, it's important to predict customer behavior to improve how well they keep customers and to make their marketing plans better. This project thoroughly focuses on predicting if all customers will purchase health insurance according to their age, in addition to salary estimates. The project is wide-ranging. We use machine learning classification algorithms to analyze the dataset and spot patterns. The primary goal involves identifying a universally relevant model with maximal precision for this predictive task. Insurance companies can use this study's results to help them make data-directed decisions and improve how they engage customers.

Literature Review

In the past, multiple studies have examined how machine learning can predict all customer actions in insurance. For example:

Logistic Regression is used a lot for binary classification tasks because it is simple and easy to understand.

- Random Forest has demonstrated superior performance, including a number of instances, in its handling of non-linear relationships as well as in avoiding overfitting.
- SVM is particularly effective in truly high-dimensional spaces; however, it still is computationally expensive.
- Decision Trees are fully interpretable, but without sufficient tuning, they often tend to overfit.

This project uses all of these improvements as a base. One real-world dataset is then used to compare each algorithm's performance.

Problem Statement

The problem is to predict whether a customer will purchase health insurance based on their age and estimated salary. The dataset contains user details, excluding personal data such as passwords and account numbers. The challenge is to select and evaluate the most suitable classification algorithm that balances accuracy and generalization.

Data Collection and Preprocessing

- **Dataset:** The dataset includes customer attributes such as age and estimated salary.
- **Preprocessing Steps:**
 - Handle missing values (e.g., impute missing salaries).
 - Normalize numerical features (e.g., scale age and salary to a range of 0 to 1).
 - Split the dataset into training and testing sets (e.g., 80% training, 20% testing).

Methodology

We employ the following classification algorithms:

1. **Logistic Regression:** A linear model for binary classification.
2. **K-Nearest Neighbours (KNN):** A non-parametric algorithm that classifies based on similarity.
3. **Support Vector Machine (SVM):** A powerful algorithm for high-dimensional data.
4. **Decision Trees:** A tree-based model for interpretable predictions.
5. **Random Forest:** An ensemble method that combines multiple decision trees. We evaluate each model using metrics such as accuracy, precision, recall, and F1-score.

Implementation

Code Screenshots:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns

data = pd.read_csv('insurance.csv')

data = data[['age', 'charges']].dropna()
median_charges = data['charges'].median()
data['Purchased'] = (data['charges'] > median_charges).astype(int)

X = data[['age', 'charges']]
y = data['Purchased']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

models = {
    'Logistic Regression': LogisticRegression(),
    'KNN': KNeighborsClassifier(),
    'SVM': SVC(),
    'Decision Tree': DecisionTreeClassifier(),
    'Random Forest': RandomForestClassifier()
}
```

```

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    results[name] = {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1': f1_score(y_test, y_pred)
    }

results_df = pd.DataFrame(results).T
print("Model Performance Metrics:")
print(results_df)

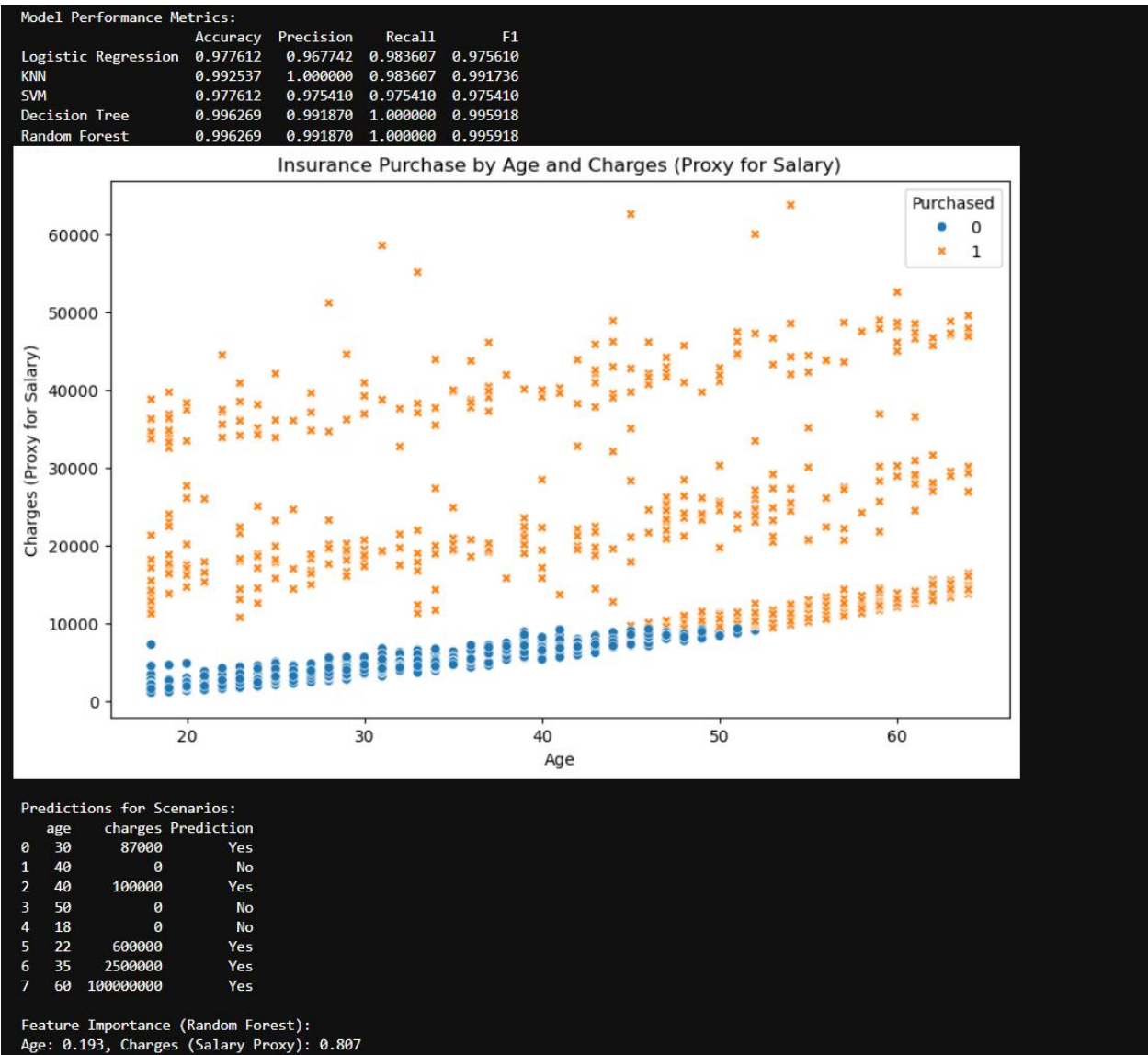
plt.figure(figsize=(10, 6))
sns.scatterplot(data=data, x='age', y='charges', hue='Purchased', style='Purchased')
plt.title('Insurance Purchase by Age and Charges (Proxy for Salary)')
plt.xlabel('Age')
plt.ylabel('Charges (Proxy for Salary)')
plt.show()

test_scenarios = pd.DataFrame([
    'age': [30, 40, 40, 50, 18, 22, 35, 60],
    'charges': [87000, 0, 100000, 0, 0, 600000, 2500000, 100000000]
])
test_scaled = scaler.transform(test_scenarios)
best_model = models['Random Forest']
predictions = best_model.predict(test_scaled)
test_scenarios['Prediction'] = predictions
print("\nPredictions for Scenarios:")
print(test_scenarios.replace({'Prediction': {0: 'No', 1: 'Yes'}}))

importances = models['Random Forest'].feature_importances_
print("\nFeature Importance (Random Forest):")
print(f"Age: {importances[0]:.3f}, Charges (Salary Proxy): {importances[1]:.3f}")

```

Output Screenshot:



Conclusion

This project shows how well machine learning can predict whether customers will buy insurance. Out of all the methods tried, Random Forest stood out as the winner—it nailed high accuracy without tripping over itself by overcomplicating things. What we learned here could really help insurance companies fine-tune their marketing game and connect better with their customers.

References

- Scikit-learn from <https://scikit-learn.org>
- Youtube : Sklearn implementation
- Insurance Dataset from [kaggle.com](https://www.kaggle.com)